COMP3311 Week 2 Thursday Lecture

- Week 02 Thursday
- Assignment 1 Database Schema
- Assignment 1 Database Schema (cont)
- Exercise: ER-to-SQL (2)
- Exercise: ER-to-SQL (3)
- Modifying PostgreSQL databases
- PostgreSQL Databases
- psql
- psql (cont)
- Exercise: Creating a database
- Populating a Database
- Exercise: Inserting Tuples
- Exercise: More Inserting Tuples
- Updating Tuples
- Deleting Tuples
- Deleting Tuples (cont)
- Another Example Database: Bars&Beers/Drinkers
- Exercise: Delete/Update Examples
- Exercise: Joins
- Extra/Revision

◆ Week 02 Thursday

In today's lecture ...

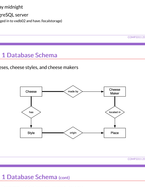
- Assignment 1 DB preview
- More ER → SQL and SQL DDL
- Building/modifying/restoring a database
- Assignment 1 released on Saturday, due end week 5

Things to do ...

- Quiz before Friday midnight
- Set up your PostgreSQL server
  (SQL students have logged in to vlad02 and have .localacnet)

◆ Assignment 1 Database Schema

Database about cheeses, cheese styles, and cheese makers



◆ Assignment 1 Database Schema (cont)

Details of entities, with example data (does not include all entities):

```
Cheeses(id, name, style, made_by, aged_for, notes)
(123, 'Stokes Point', 456, 321, '4 months', 15, 'smoked')

Makers(id, name, founded, located_in)
(321, 'King Island Dairy Co', 1902, 421)

Styles(id, name, origin, hardness, milk, mix_aging, max_aging)
(456, 'Cheddar', 234, 'semi-hard', 'cow', '2 months', '24 months')

Places(id, country, region, town)
(234, 'UK', 'Gloucestershire', 'Cheddar')
(421, 'Australia', 'King Island', 'Grassmere')
```


◆ Exercise: ER-to-SQL for Cheese Database

Convert the cheese ER data model to an SQL schema

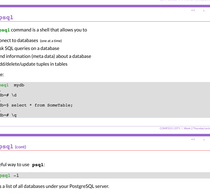
- assume that each entity has (at least) attributes **id** and **name**

◆ Exercise: ER-to-SQL (2)

Convert this ER design to SQL:

◆ Exercise: ER-to-SQL (3)

Convert the Publishing ER model to SQL



- A TFN is stored as a 9-digit number
- An ABN is stored as an 11-digit number
- An ISBN (10-digit version) looks like 978-3-16-148410-0

◆ Modifying PostgreSQL databases

PostgreSQL commands for manipulating databases

```
dropdb DatabaseName
createdb DatabaseName
pg_dump DatabaseName > NewDumpFile
psql DatabaseName [ -f FileOfQuitStatements ]
```

SQL commands for changing data in a database from within **psql**

```
insert into Table values ('Val1','Val2', ...);
delete from Table where Condition
update Table set Field = Value where Condition
create table Table ( Attributes/Constraints )
drop table Table
copy Table from ...  -- Postgres/Oracle
```


◆ PostgreSQL Databases

Create a database in PostgreSQL via

```
$ createdb DatabaseName
```

Creates an empty database (no schema, no data)

Remove a database in PostgreSQL via

```
$ dropdb DatabaseName
```

Removes schema and all data permanently

Remove an entire PostgreSQL server (on **vlad02**)

```
$ sm -fr /local/postgr/DISTR/pgsql
```

Removes all server files, all databases, all data !

◆ psql

The **psql** command is a shell that allows you to

- connect to databases (one at a time)
- ask SQL queries on a database
- find information (meta-data) about a database
- add/delete/update tuples in tables

Usage:

```
$ psql mydb
...
mydb=# \d
...
mydb=# select * from BomeTable;
...
mydb=# \q
```


◆ psql (cont)

A useful way to use **psql**:

```
$ psql -t1
```

Gives a list of all databases under your PostgreSQL server.

The "databases" **postgres**, **template1**, **template2**

- are special databases used internally by PostgreSQL
- do not **dropdb** them

◆ psql (cont)

psql commands can span several lines, terminated by typing ;

The **psql** command has several prompts

- **dbn=#** , waiting to start an SQL statement (or psql meta-command)
- **dbn=#** , waiting for rest of SQL statement
- **dbn(#** , waiting for rest of expression (**...**)
- **dbn'#** , waiting to finish a string (expecting closing ')

Note that **dbn** will be (replaced by) the name of the current database

Note that **#** means you are super-user: normal users get **>**

psql meta-commands are single-line commands

◆ psql (cont)

psql has a range of meta-commands, beginning with \

- **help** , a quick list of useful meta-commands
- **\?** , a list of all meta-commands (very out of date)
- **\d** , list of all tables and views in current schema
- **\dt** , list of all tables in current schema
- **\d Table** , list of all attributes in *Table*
- **\df** , list of all user-defined functions in current schema
- **\q** , exit **psql**

◆ Exercise: Creating a database

On **vlad02**, do the following:

- create a database called **xyz**
- examine its (empty) schema
- within **xyz** create a table

```
create table R (
    a integer primary key,
    y float not null,
    z text
);
```

- examine the schema again
- examine the attributes of table **R**
- how many tuples are in table **R**?
- remove the database **xyz**

◆ Populating a Database

Basic way of adding tuples to a database:

```
db=# insert into Table values (val1,val2,...);
```

Adds a tuple to table **Table** assuming

- values satisfy all constraints on tuples/table

Ways it can fail ...

- value for primary key field is already in the table
- tuple has **null** values for fields defined as **not null**
- value for some field violates constraints on that field
- etc. etc. etc.

◆ Exercise: Inserting Tuples

Which **insert** statements are successful?
If successful, what tuple value is inserted?

```
create type Mood as enum ('sad','happy');

create table People (
    name text not null,
    feels Mood
);

insert into People values ('John','happy');
insert into People values ('Andrew','sad');
insert into People values ('Peter','happy');
insert into People(name) values ('John');
insert into People(feels) values ('happy');
```


◆ Exercise: More Inserting Tuples

Which **insert** statements are successful?
If successful, what tuple value is inserted?

```
create domain Positive as integer check (value > 0);

create table Points (
    x Positive default 1,
    y positive
);

insert into Points values (3,4);
insert into Points values (1,null);
insert into Points values (-3,4);
insert into Points values (0,0);
insert into Points(x,y) values (default,1);
insert into Points values (default,1);
```


◆ Updating Tuples

Existing tuples can be modified "in place":

```
update Table
set Field = Expression
where Condition
```

Note that ***Expression***

- could be a constant value
- could be an expression involving the ***Field***

Condition can match zero or more tuples

- all matching tuples will have the update applied to them

If no **where** clause, all tuples in **Table** are updated

◆ Updating Tuples (cont)

Examples:

```
-- change a student's address

update Students
set address = '256 Rainbow St., Coogee'
where id = 5544332;

-- give all white parts red

update Parts set colour='red' where colour='white'

-- give all employees a 20% raise

update Employees set salary = salary * 1.2;
```


◆ Deleting Tuples

Tuples can be permanently removed:

```
delete from Table where Condition
```

Removes all tuples from **Table** satisfying ***Condition***

If ***Condition*** is **true** (or omitted), all tuples are deleted

Be careful to precisely specify ***Condition***

If any tuple is referenced from elsewhere, deletion will fail.

◆ Deleting Tuples (cont)

Examples:

```
-- remove all red products

delete from Products where colour = 'red';

-- remove all employees in COMP3311 DB

delete from Enrolments
where exists(select * from Courses
             where Courses.id = Enrolments.course
             and Courses.dept = 'COMP3311');
```


◆ Another Example Database: Bars&Beers/Drinkers

◆ Exercise: Delete/Update Examples

```
-- Delete all drinkers who frequent more than 2 bars
delete from Drinkers
where exists(select * from Frequents
             where Frequents.drinker = Drinkers.name
             group by drinker
             having count(bar) > 2);

-- Delete all beers made by Fosters
delete from Beers
where manf = 'Fosters';

-- Increase the price of all beers by 10%
update Serves set price = price * 1.10;

-- Increase the price of all beers by 10% except for Fosters
update Serves set price = price * 1.10
where beer not in (select name from Beers
                   where manf = 'Fosters');
```


◆ Exercise: Joins

```
-- Find all drinkers who frequent bars that serve some beer they like
select distinct d.name, f.bar
from Drinkers d, Frequents f, Serves s, Drinkers_Beers db
where d.name = f.drinker
  and f.bar = s.bar
  and s.beer = db.beer
  and d.name = db.drinker
order by d.name, f.bar;

-- Find all drinkers who frequent bars that serve some beer they like
select distinct d.name, f.bar
from Drinkers d, Frequents f, Serves s, Drinkers_Beers db
where d.name = f.drinker
  and f.bar = s.bar
  and s.beer = db.beer
  and d.name = db.drinker
order by d.name, f.bar;
```

```
where  course= COMP4321  and term= 25T1

-- sack everybody (the Elon Musk solution)

delete from Employees;
```


❖ Exercise: Yet More Modifying Tuples

Consider the following SQL schema:

```
create table Student (
    id      serial primary key,
    name    text not null,
    degree  numeric(4)
);
create table Course (
    id      integer primary key,
    code    char(8) check (code ~ '[A-Z]{4}[0-9]{4}'),
    term    char(4) check (term ~ '[0-9]{2}T[0123]')
);
create table Enrolment (
    stuID   integer references Student(id),
    crsID   integer references Course(id),
    mark    integer check (mark between 0 and 100),
    grade   char(2)
            check (grade in ('HD','DN','CR','PS','FL'))
);
```

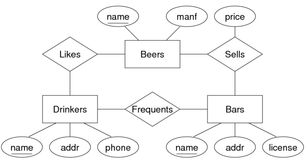

❖ Exercise: Yet More Modifying Tuples (cont)

State what the effect of each of these statements would be:

```
-- Student(id,name,degree),  Course(id,title,term)
-- Enrolment(stuID,crsID,grade,mark)

-- assume 'student_id_seq' has value 5654320
insert into Student(name,degree) values ('John',3778);
insert into Student(name,degree) values ('Jack',3778) returning id;
insert into Student(id,name,degree) values (default,'Bill',3778);

insert into Course values (987, 'COMP3311', '25T1');

insert into Enrolment(stuID,crsID) values(5654321, 987);

-- update mark for one student
update Enrolment set mark=75, grade='DN' where stuID = 5654321;
-- adjust marks for all students in the course
update Enrolment set mark=mark*1.1 where crsID = 987;

-- a student drops the course
delete from Enrolment where stuID = 5554321 and crsID = 987;
```


❖ Another Example Database: Beers/Bars/Drinkers

Consider the following ER model:



Define an SQL schema for this database

❖ Exercise: Delete/Update Examples

Give SQL statements to carry out the following actions:

- Insert a new beer "Jumping the Shark" from MoonDog
- Sell "Jumping the Shark" for $5.00 at Marble Bar
- Increase the prices for all beers by 10%
- Remove all drinkers who live in Randwick
- Remove all beers selling for more than 5.00
- Close down the Marble Bar (it really did close)

❖ Bulk Insertion

Entering tuples interactively one-by-one is not feasible

Alternative: put **`insert`** statements in a file and run

```
$ psql DatabaseName -f FileName
```

Attempts to execute each **`insert`** statement:

- if tuple valid, inserted into database
- if tuple not valid, prints error message, then continues

Note that ***`FileName`*** can contain any SQL statements

Might consist only of **`create table`** statements to build a schema

❖ Bulk Insertion (cont)

A common way of building a database

```
$ createdb mydb
$ psql mydb -f schema.sql
$ psql mydb -f data.sql
```

where

- **`schema.sql`** contains table and type definitions
- **`data.sql`** contains statements to insert tuples

❖ Bulk Insertion (cont)

How I "debug" a database schema

```
$ dropdb mydb
$ createdb mydb
$ psql mydb -f schema.sql > .errs 2>&1
$ vi .errs
  # fix any errors that appear in .errs
$ psql mydb -f data.sql > .errs 2>&1
$ vi .errs
  # fix any errors that appear in .errs
```

Repeat until **`.errs`** contains no lines with **`ERROR`**

❖ Bulk Insertion (cont)

Alternative way of inserting tuples

```
copy TableName ( attribute names ) from stdin;
... lines containing tab-separated values ...
... one value for each of the named attributes ...
\.
```

Difference between **`copy`** and multiple **`insert`**s

- with **`insert`** ...
  - all tuples with valid values are inserted
  - tuples with invalid values are not inserted
- with **`copy`** ...
  - if any tuple contains invlid values, nothing is inserted

❖ Dump/Restore

Once a database is built, can make a complete copy in a text file

- the whole schema (including types, constraints, etc), and all data

by running the command

```
$ pg_dump -O -x DatabaseName > DumpFileName
```

and can make a new copy via

```
$ createdb newdb
$ psql newdb -f DumpFileName
```

We generally supply databases using pre-built dump files